

6.2 Contratos: gerar contrato a partir de proposta aprovada — status de entrega

Bloco 6.2 Contratos: gerar contrato a partir de proposta aprovada · Sprint 6 — Contratos · Data: 2026-09-23 · Branch: `main`
· Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Bloco concluído. Proposta aprovada ganhou o botão **Gerar contrato**, que abre um formulário já preenchido com os dados da proposta. Ao salvar, o contrato é criado e os itens são copiados numa transação só. A migration do bloco, `20260923162000_gerar_contrato_de_proposta.sql`, já tinha sido aplicada em gc-dev na abertura da sprint (ver 6.1, seção 5). `migration list` : **23/23 alinhadas**.

1. As seis atividades do bloco

ATIVIDADE DO CLICKUP	ESTADO
Botão 'Gerar contrato' na tela da proposta, visível só quando status='aprovada'	<code>detail-header.tsx</code> ; só admin e comercial. A rota <code>/propostas/[id]/gerar-contrato</code> redireciona se a proposta não estiver aprovada ou se o perfil for outro
Formulário pré-preenchido com valores, pct_* e condições da proposta	<code>propostaToContratoFormValues</code> : descrição, valor, desconto, os quatro percentuais, condições e observação. A data de assinatura vem como hoje e o número em branco
Copiar todos os itens da proposta para o contrato, respeitando o XOR	caixa "Copiar os N itens", marcada por padrão. A cópia leva <code>contrato_id</code> e <code>proposta_id</code> nulo, e a proposta continua com os itens dela
Gravar proposta_origem_id no contrato	pela função; a obra é sempre a da proposta, como exige a FK <code>contratos_proposta_fk</code>
Transação: ou copia tudo ou não copia nada	função <code>gerar_contrato_de_proposta</code> : o insert do contrato e o dos itens estão na mesma transação
Impedir gerar dois contratos da mesma proposta sem confirmação explícita	a função recusa sem <code>p_confirmar_duplicado</code> ; a tela avisa antes de enviar e abre um diálogo de confirmação

Arquivos: `src/app/(app)/propostas/[id]/gerar-contrato/{page,gerar-form}.tsx` e `actions.ts` (`gerarContratoDeProposta`), mais o botão em `detail-header.tsx` . O formulário é o `contratos/contrato-form.tsx` , o mesmo que o 6.3 vai usar para o contrato avulso.

2. Decisões

2.1 A transação mora no banco

Pelo PostgREST seriam duas chamadas, uma para inserir o contrato e outra para os itens. Se a segunda falhasse, sobraria um contrato sem itens. Uma função plpgsql roda inteira numa transação. Ela também confere as regras por conta própria (proposta aprovada, duplicado, RLS), porque o PostgREST a expõe direto, sem passar pela action. Há passo provando que o visualizador chamando a função direto no banco não cria nada.

2.2 Itens copiados, não movidos

O XOR proíbe um item ligado aos dois, e a proposta aprovada é o registro do que foi vendido, por isso continua com os itens dela. A cópia leva todos os campos: número, tipo, descrição, linha, acabamento, dimensões, quantidade, unidade, valor unitário, localização, vidros e observação. Há passo comparando campo a campo. Duas coisas ficam de fora:

- **a foto:** `foto_url` é o path de um arquivo só. Com as duas linhas apontando para ele, excluir uma apagaria a foto da outra (mesmo motivo do duplicar do 5.7). A tela avisa que as fotos não vão junto;
- **o autor:** `created_by` do item copiado é quem gerou o contrato.

2.3 Com itens, o valor do contrato é a soma deles

Com a cópia marcada, o campo Valor total fica travado na soma dos itens, e a função grava a soma, seja qual for o valor que chegou no corpo da requisição. Sem a cópia, vale o valor digitado. O desconto vem da proposta. Se ele passar da soma, a geração é recusada inteira, sem contrato nem item pela metade, e há passo provando isso.

2.4 "Duplicado" é outro contrato vigente da mesma proposta

A função trava a linha da proposta (`for update`) antes de conferir, então dois cliques simultâneos não passam os dois. **Contrato rescindido não conta:** gerar outro depois de uma rescisão é o caso normal, e não pede confirmação. A tela avisa o duplicado antes de enviar, com os números dos contratos existentes. Se alguém gerar um contrato enquanto o formulário está aberto, a recusa do servidor devolve os números e abre a mesma confirmação.

2.5 Depois de gerar, a listagem filtrada

O detalhe do contrato é do bloco 6.4. Até lá, gerar leva a `/contratos?busca=<número>`, com o contrato novo na tela. O 6.4 troca esse destino pelo detalhe.

2.6 O número não é copiado

Contrato e proposta têm numerações próprias. Copiar levaria a pessoa a salvar o número da proposta como número do contrato. Número repetido é recusado com "Já existe um contrato com esse número".

3. Números reais do `npm run validar`

Rodado em 2026-09-23 contra gc-dev, na cópia isolada do working tree (`.next` próprio, portas 3311/9422).

CAMADA	ANTES DO 6.2 (FIM DO 6.1)	AGORA	
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint</code> <code>warnings or errors</code>	ok
unitário	189 casos	189 casos, 0 falhas	=
build	34 rotas	35 rotas, iguais à baseline	+1 (<code>/propostas/[id]/gerar-contrato</code>)
runtime	69/69 rotas	76/76 rotas ok	+7
dados	25/25 checagens	26/26 checagens ok	+1
escrita	160/160 passos	177/177 passos ok	+17
navegador	67/67 passos	73/73 passos ok , console limpo	+6

A baseline de rotas foi regravada com `validar.sh build --aceitar-rotas`. O diff foi exatamente uma linha, `/propostas/[id]/gerar-contrato`.

Sem caso unitário novo: as regras do bloco estão na função do banco e nos helpers do form. `propostaToContratoFormValues` é conferido pelo navegador (o form nasce com os valores certos) e não por `node --test`. Isso fica registrado como pendência (seção 4).

As 7 rotas do runtime: a proposta aprovada mostra "Gerar contrato" (e não "Mudar status") · a enviada não mostra · o visualizador não vê o botão · o form abre com a obra travada e "A proposta não tem itens" · o comercial abre o form · o visualizador leva 307 · proposta que não está aprovada leva 307.

A checagem de dados: a query da page que lista os contratos vigentes gerados de uma proposta. Voltou 3 linhas, os `TESTE-FASE2-*` da automação, que vêm da mesma proposta.

Os 17 passos da escrita (15 do fluxo e 2 de limpeza): proposta de teste criada · em rascunho não gera · aprovada · visualizador recusado pela action · **visualizador chamando a função direto no banco não cria contrato** · admin gera com cópia · contrato ativo, da obra da proposta, com `proposta_origem_id` · valor = soma (650), desconto 100, final 550, pct e prazo do form · **os 2 itens copiados campo a campo** · XOR (a cópia só tem `contrato_id` e a proposta mantém os seus) · **segundo contrato sem confirmação é recusado e devolve o número do existente** · **desconto acima da soma recusado sem deixar contrato nem item** · número repetido recusado · com confirmação, gera o segundo sem cópia (700, nenhum item) · com os anteriores rescindidos, comercial gera sem confirmação · limpeza dos 3 contratos com os itens · limpeza da proposta.

Os 6 passos do navegador, sobre a PROP-2026-008 do seed (aprovada, desconto de R\$ 8 mil): o seed existe · o botão abre o form já preenchido (valor 178.000, desconto 8.000, sinal 30%, número em branco, obra travada) · gerar leva à listagem com o contrato novo · na segunda vez, o form avisa com o número do contrato existente · sem confirmar, o segundo contrato ainda não existe · confirmado, os dois existem ligados à proposta e com o desconto dela. Screenshots: `19-gerar-contrato-form.png` e `20-gerar-contrato-confirmar.png`.

Os dois contratos que o navegador gera saem no `finally` da camada. Conferido em gc-dev depois das rodadas: 0 contratos e 0 propostas `RUN-` / `VALIDA-` sobrando, 0 itens órfãos, 0 contratos ligados à PROP-2026-008.

3.1 O que as rodadas pegaram

- **Passo do navegador instável:** com a página aberta pela URL, o número preenchido antes da hidratação do React era apagado quando o `react-hook-form` montava, e o envio parava em "Número obrigatório". O passo agora repreenche até o valor ficar. Não é defeito da tela: uma pessoa não digita antes de a página hidratar.
- **Ordem do formulário:** a seção "Itens e origem" aparecia abaixo da nota de Anexos. Subiu para antes dela. Screenshot conferido.

4. O que não foi validado

- **A caixa "Copiar itens" pela tela:** a proposta aprovada do seed não tem itens, então o navegador nunca viu a caixa marcada, o Valor total travado na soma nem o desmarcar. A cópia em si está provada pela escrita (a action com `copiarItens`). Falta uma proposta aprovada com itens no seed. Fica para o 6.6 ("seed_contratos.sql").
- **Dois cliques simultâneos em "Gerar":** a trava `for update` está na função, mas nenhum passo dispara duas gerações em paralelo. O 5.6 mediu corrida no trigger e o 5.7 na troca de números; esta não foi medida.
- **O caminho da recusa pelo servidor na tela:** alguém gerar um contrato enquanto o form está aberto. A escrita prova que a action devolve `contratosExistentes`, mas o navegador só exercitou o aviso que já vem com a página.
- **`propostaToContratoFormValues` sem teste unitário:** ele fica num arquivo de rota que importa zod; só o navegador o confere.
- **Destino depois de gerar:** é a listagem filtrada, não o detalhe, que só existe no 6.4.

5. Registro para quem for operar gc-prod

`20260923162000_gerar_contrato_de_proposta.sql` não foi aplicada em prod. Ela cria só uma função e não mexe em dado. Depende da `20260923160000` (colunas `desconto` e `historico` em `contratos`), então entra depois dela. A ordem completa está no documento do 6.1, seção 6.